ROLE ASSIGNMENT

Fullstack Engineer

Ruby on Rails + React · Take-Home Guide

FORMAT	DURATION	FOCUS
Paid take-home	4 hours	API · rewards · React UI

You are already in the correct repository. This branch (main) contains only the process guide — switch branches for assignment artifacts.

Start here

This assignment mirrors institutional staking product work: own a vertical slice across Rails APIs and a React application — portfolio balances, rewards reporting, and stake operation workflows — with chain complexity behind a service boundary.

- Branch: main — this guide only
- Choose scratch (from zero) or boiler-plate (starter stubs)

Repository layout

Use these branches in the same repository to complete the assignment.

```
fullstack-rails/
+-- main                 <- you are here
|   +-- HOME_TASK_GUIDE.pdf
+-- scratch              <- full requirements (build your own)
|   +-- README.md
+-- boiler-plate         <- Rails API + React starter
    +-- backend/  frontend/  README.md
```

IMPORTANT

If you still only see this PDF in the "fullstack-rails" folder, you are on main. Switch branches to continue.

01 Purpose

Deliver a scoped Institutional Staking Console: a Rails service that models organizations, networks, validators/delegations, and rewards; abstracts a mock chain client; exposes secure REST APIs; and a React UI for portfolio, rewards rollups, and stake request status.

02 Process

1. Read this guide
2. Checkout scratch or boiler-plate
3. Implement API + UI + background sync slice
4. Add tests and document run steps / assumptions
5. Submit your branch / fork

```
git checkout scratch
# or
git checkout boiler-plate
```

03 Product summary

- Portfolio view: staked balance + total rewards by network
- Rewards reporting: daily rollups with filters
- Stake operations: create request, track pending/confirmed/failed
- Background sync: ingest rewards from a mock chain adapter

04–05 Evaluation & guidelines

What we evaluate

- Rails API & data modeling
- Service boundary vs chain SDK
- Rewards/portfolio correctness
- React UX for ops workflows
- Tests + pragmatic security

Guidelines

- 4-hour timebox
- Rails + React/TS required
- Amounts as integer strings
- No real mainnet keys
- Simplicity over framework sprawl

06 Submission & compensation

- Working backend + frontend for the required slice
- README with install/run/test commands
- Assumptions, open questions, and time spent

Paid take-home. Compensation applies whether or not we proceed.